SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 40%

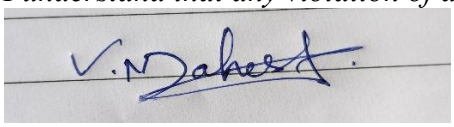
QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I MAHESH V(192524026) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

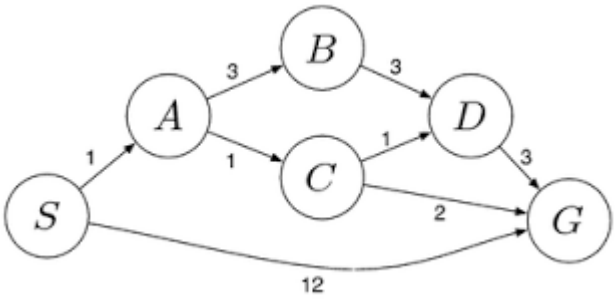
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Question 1:
The Water Jug Problem Question:
Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Comprehensive Answer:

To solve this problem systematically using Artificial Intelligence principles, we must first formulate it as a state space search problem. In AI, a state space represents all the possible configurations a given system can be in. For the Water Jug problem, we represent the state space mathematically as an ordered pair (x, y). In this formulation, 'x' represents the current volume of water held in the 4-gallon jug, and 'y' represents the current volume of water held in the 3-gallon jug.
Because of the physical limitations of the jugs, we have strict state constraints. The value of 'x' must always be an integer between 0 and 4, inclusive. The value of 'y' must always be an integer between 0 and 3, inclusive. Therefore, our initial state, where both jugs begin completely empty, is represented as (0, 0). Our objective is to reach a specific goal state. According to the problem prompt, we must get exactly 2 gallons of water into the 4-gallon jug. Thus, our goal state is defined as (2, y), where 'y' can be any valid volume, though in the most optimal sequence, 'y' will typically end up being 0, resulting in the state (2, 0).
Next, we must define the production rules, or the set of valid operators (actions) that transition our system from one state to another. These are strictly derived from the explicit assumptions provided.

- Rule 1: If the 4-gallon jug is not full ($x < 4$), we can fill it from the pump, changing the state from (x, y) to (4, y).
- Rule 2: If the 3-gallon jug is not full ($y < 3$), we can fill it from the pump, changing the state from (x, y) to (x, 3).
- Rule 3: If the 4-gallon jug has water ($x > 0$), we can empty it onto the ground, changing the state from (x, y) to (0, y).
- Rule 4: If the 3-gallon jug has water ($y > 0$), we can empty it onto the ground, changing the state from (x, y) to (x, 0).
- Rule 5: We can pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is completely full. This rule applies if $x + y >= 4$ and $y > 0$. The resulting state is (4, y - (4 - x)).
- Rule 6: We can pour water from the 4-gallon jug into the 3-gallon jug until the 3-gallon jug is completely full. This applies if $x + y >= 3$ and $x > 0$. The resulting state is (x - (3 - y), 3).
- Rule 7: We can pour all the water from the 3-gallon jug into the 4-gallon jug without overflowing it. This applies if $x + y <= 4$ and $y > 0$. The resulting state is (x + y, 0).
- Rule 8: We can pour all the water from the 4-gallon jug into the 3-gallon jug without overflowing it. This applies if $x + y <= 3$ and $x > 0$. The resulting state is (0, x + y).

By applying these rules, we can trace the most efficient pathway to the solution step-by-step:

- Step 0: We begin at the Initial State (0, 0). Both the 4-gallon and 3-gallon jugs are completely empty.
- Step 1: We apply Rule 2. We fill the 3-gallon jug directly from the pump. The new state is (0, 3).

- Step 2: We apply Rule 7. We pour all 3 gallons of water from the 3-gallon jug into the 4-gallon jug. The 3-gallon jug is now empty, and the 4-gallon jug contains 3 gallons. The new state is (3, 0).
- Step 3: We apply Rule 2 again. We refill the 3-gallon jug directly from the pump. The 4-gallon jug still holds 3 gallons. The new state is (3, 3).
- Step 4: We apply Rule 5. We carefully pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is entirely full. Because the 4-gallon jug already had 3 gallons, it only takes 1 gallon to fill it. This leaves exactly 2 gallons of water remaining in the 3-gallon jug. The new state is (4, 2).
- Step 5: We apply Rule 3. We take the 4-gallon jug, which is currently full, and pour all of its contents out onto the ground, emptying it completely. The 3-gallon jug remains untouched with its 2 gallons. The new state is (0, 2).
- Step 6: We apply Rule 7 for the final time. We pour all the remaining water (which is exactly 2 gallons) from the 3-gallon jug into the empty 4-gallon jug. The new state is (2, 0).

We have successfully reached the goal state of (2, y), specifically (2, 0). The 4-gallon jug now contains exactly 2 gallons of water, and the problem is successfully solved without the use of any external measuring devices.

Question 2:

Mars Rover Agent Analysis Question:

Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions: What are the percept's for this agent? Characterize the operating environment. What are the actions the agent can take? How can one evaluate the performance of the agent? What sort of agent architecture do you think is most suitable for this agent? Why?

Comprehensive Answer:

To thoroughly understand how a Mars Rover operates autonomously millions of miles from Earth, we must analyze it through the lens of artificial intelligence as an intelligent agent. The first step in this process is establishing the PEAS framework, which stands for Performance measure, Environment, Actuators, and Sensors.

Performance Measure: Evaluating the performance of the Mars Rover agent requires looking at a multitude of critical success factors. The primary goal is the maximization of high-value scientific data collection. This involves discovering evidence of past water, analyzing geological structures, and searching for biological markers. However, data collection must be balanced against survival. Therefore, the performance measure heavily penalizes negative outcomes like tip-overs, getting stuck in deep sand, crashing into boulders, or draining the battery below critical operational thresholds. Distance traversed, accuracy of spectral analyses, and the sheer volume of data successfully transmitted back to NASA control on Earth are all quantifiable performance metrics.

Operating Environment Characterization:

The operating environment of the Mars Rover is extreme, hostile, and complex. In AI terminology, we characterize it across several dimensions:

- Partially Observable: The rover does not possess a complete, omniscient view of the planet. It can only perceive what is immediately in front of its cameras and sensors. Features like hidden drop-offs or incoming dust storms are obscured, meaning the agent must make decisions based on incomplete information.

- Stochastic (and Non-deterministic): When the rover attempts an action, the outcome is never 100% guaranteed. If it commands its wheels to drive forward one meter, wheel slippage on loose Martian soil might mean it only travels 0.8 meters.
- Sequential: The environment is highly sequential. Every action the rover takes fundamentally alters the options available in the future. Deciding to drive down a steep crater wall might secure great scientific samples, but it sequentially limits the rover's ability to drive back out later.
- Dynamic: Mars is a changing world. While the bedrock stays still, the environment features shifting winds, massive dust storms that block solar panels, and severe temperature fluctuations between day and night cycles.
- Continuous: Unlike a chessboard with discrete squares, Mars is a continuous space. The coordinates, the angle of the sun, the battery voltage, and the temperature are all continuous mathematical variables that the agent must process.

Actuators (Actions):
The actuators are the physical mechanisms the agent uses to interact with its environment and execute commands. For a Mars Rover, these include a complex drive system (usually a rocker-bogie suspension with independently driven wheels allowing for forward, backward, and turning motions). Additional actuators include highly articulated robotic arms designed to place scientific instruments directly onto rock surfaces. Furthermore, it has drilling mechanisms to core into rocks, scoops for soil retrieval, heating elements to survive the cold Martian nights, and steerable high-gain antennas to beam data directly to orbital relays or Earth.
Sensors (Percepts):
The agent gathers data through a vast array of specialized sensors. Vision is achieved through stereoscopic navigation cameras (Navcams) for 3D pathfinding, hazard avoidance cameras (Hazcams) mounted low on the chassis, and high-resolution panoramic cameras. For chemical and geological analysis, it uses spectrometers (like alpha-particle X-ray spectrometers) and microscopic imagers. Internally, the rover must sense its own state. It utilizes wheel odometers to measure distance, inertial measurement units (IMUs) containing accelerometers and gyroscopes to detect its tilt and prevent flipping, and internal thermometers and voltmeters to monitor vital subsystem health.

Suitable Agent Architecture:
The most suitable agent architecture for a Mars Rover is a hybrid consisting of a Utility-Based Agent integrated with a Learning Architecture. Why is this the case? A simpler Goal-Based agent only cares about reaching a predefined destination (e.g., "Drive to Crater X"). It evaluates paths as a binary: does this path lead to the goal, yes or no? However, on Mars, the safety and efficiency of the journey are paramount. A Utility-Based agent applies a mathematical utility function to all possible states. It can weigh trade-offs. It understands that Path A might be the shortest route to Crater X, but it is covered in sharp rocks that could damage the wheels (low utility). Path B is longer but consists of smooth bedrock and better angles for the solar panels to charge (high utility). The agent will actively select the path that maximizes its overall "happiness" or utility score, thus ensuring survival while achieving scientific goals. Furthermore, a learning component is strictly necessary because the environment is too stochastic to hard-code every possible scenario from Earth. If the rover notices one of its wheels is drawing too much current (indicating wear), the learning agent can update its internal model of the world and adjust future utility calculations to avoid steep inclines, adapting on the fly to survive.

Question 3:

Formulating the 8-Queens Problem Question:
Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Comprehensive Answer:

The 8-Queens problem is a foundational problem in artificial intelligence, serving as the quintessential example of a Constraint Satisfaction Problem (CSP) and heuristic search methodologies. The fundamental objective is to place exactly eight chess queens on a standard 8x8 chessboard in such a configuration that no two queens threaten each other. Given the movement rules of a queen in chess, this means that no two queens can share the same row, the same column, or the same diagonal line. The problem prompt explicitly notes that simply placing queens on the board is insufficient; any configuration where queens attack each other (such as a first-column queen sharing a diagonal with a last-column queen) is an invalid solution.
To explore the search space, we must formally define the problem components. There are multiple ways to formulate this.

Naive Problem Formulation:

- States: Any arrangement of 0 to 8 queens on the 64 available squares of the chessboard.
- Initial State: A completely empty 8x8 chessboard.
- Actions (Operators): Add a single queen to any empty square on the board.
- Goal Test: Exactly 8 queens are on the board, and no single queen is in a position to attack any other queen. The issue with this naive formulation is the massive size of the search space. If we blindly place 8 queens on 64 squares, there are "64 choose 8" possible combinations, which equates to roughly 4.4 billion possible states. Evaluating all of these using uninformed search would be computationally inefficient.

Incremental Problem Formulation (Optimized):

Since we know by definition that no two queens can exist in the same column without attacking each other, we can heavily restrict our state space by enforcing a rule: we will only ever place one queen per column.

- States: Arrangements of 'k' queens (where 'k' is a number between 0 and 8), with exactly one queen placed in each of the first 'k' columns, and crucially, none of these placed queens are attacking each other.
- Initial State: An empty 8x8 chessboard.
- Actions: Add a queen to the leftmost empty column, but only in a row where it is not currently being attacked by any of the previously placed queens on its left.
- Goal Test: 8 queens are successfully placed on the board without any conflicts. By enforcing the one-queen-per-column rule, the search space drops immediately from 4.4 billion down to 8^8 (approximately 16.7 million states). When we further enforce the rule of not placing a queen in a row or diagonal that is already under attack during

the generation phase, the state space of valid partial configurations shrinks drastically to just 2,057 states.

Exploring the Search Space using Search Strategies:

Given this formulation, uninformed Breadth-First Search (BFS) is highly unsuitable. BFS would evaluate every possible configuration of 1 queen, then all combinations of 2 queens, keeping them all in memory. This would rapidly cause a memory overflow before reaching the solution depth of 8.
The most effective classical strategy is Depth-First Search (DFS) combined with Backtracking. Under this strategy, the AI places a queen in the first safe row of the first column. It then moves deep into the search tree, moving to the second column and placing a queen in the first safe row it finds. It proceeds column by column. The key to this strategy is the backtracking mechanism. If the AI reaches, for example, the 6th column, and realizes that every single row in that column is currently under attack by the queens in columns 1 through 5, it recognizes a "dead end." It does not waste time searching further. Instead, it "backtracks" to the 5th column, removes the queen from its current square, and tries the next available safe square in that 5th column before moving forward again. This systematic pruning of the search tree makes DFS highly memory efficient and fast for solving the 8-Queens puzzle.
Alternatively, modern AI utilizes Local Search Algorithms, specifically the Min-Conflicts Heuristic. Unlike incremental formulation, this starts with a complete-state formulation: put 8 queens on the board, one in each column, but completely ignore row and diagonal constraints (they will start out attacking each other). The algorithm then calculates a heuristic cost: the total number of attacking pairs on the board. In each iteration, the AI randomly selects one queen that is currently involved in a conflict. It evaluates every square in that queen's column and moves the queen to the square that minimizes the number of total conflicts on the board (hence, min-conflicts). This hill-climbing approach is stunningly fast. It can often solve not just the 8-Queens problem, but the 1,000,000-Queens problem, in near-constant time, proving that search strategy selection is just as important as problem formulation.

Question 4:
OLA Cab Booking Application Question:
A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Comprehensive Answer:
To build an intelligent system that fulfills a ride-sharing request on platforms like OLA, we must analyze the user's requirements. The prompt explicitly states that the customer selects cab types like mini, micro, sedan, or prime "based on his comfort" to reach the destination. This phrase is the critical indicator. A standard Goal-Based Agent is insufficient here. A goal-based agent only evaluates states based on whether they achieve the final goal (reaching the destination). To a goal-based agent, an incredibly expensive, slow ride in a cramped car is treated exactly the same as a cheap, fast ride in a luxury car, so long as both arrive at the final location.
Therefore, the appropriate problem-solving agent is a Utility-Based Agent. A utility-based agent maps complex states to a real number that represents the "degree of happiness" or satisfaction of the user. Because the user is making decisions based on comfort, budget, and time, the agent must evaluate the trade-offs between different cab types. A Prime sedan might

offer high comfort but at a higher financial cost and potentially a longer wait time. A Shared cab is cheap but has low comfort and high time cost. The utility-based agent evaluates all nearby cabs, calculates a utility score for each based on the user's specific weighted preferences, and selects the one that maximizes overall user satisfaction.

Agent PEAS Description:

- Performance: Minimize wait time, minimize fare cost, maximize passenger comfort, and successfully reach the target destination.
- Environment: The digital GPS mapping system, real-time traffic grids, available fleet locations, and the mobile app interface.
- Actuators: Dispatching cab signals, displaying ETAs and prices to the user on the screen, confirming the booking, and processing digital payments.
- Sensors: User smartphone GPS, user input (source, destination, cab category preference), and API feeds from drivers and traffic servers.

Question 5:
Uniform Cost Search (UCS) for Logistics Routing Question:
A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm. The delivery network is represented as a weighted graph.

Data extracted from the provided network graph:

- Path S to A has a cost of 1.
- Path S to G has a cost of 12.
- Path A to B has a cost of 3.
- Path A to C has a cost of 1.
- Path B to D has a cost of 3.
- Path C to D has a cost of 1.
- Path C to G has a cost of 2.
- Path D to G has a cost of 3.
-

Comprehensive Answer:

To solve this logistics routing problem, we must apply the Uniform Cost Search (UCS) algorithm. UCS is a highly reliable, optimal, and complete uninformed search strategy used for traversing weighted graphs or trees. Unlike Breadth-First Search (which simply expands the shallowest nodes) or Depth-First Search (which blindly dives down a single branch), Uniform Cost Search operates strictly on path cost. It evaluates nodes based on g(n), which is the total accumulated cost to reach a node 'n' from the start node. The algorithm utilizes a priority queue, referred to as the Frontier, to organize nodes. The node with the absolute lowest accumulated cost g(n) is always positioned at the front of the queue to be expanded next.
We will trace the execution of the algorithm step-by-step through the provided delivery network graph to find the minimum cost path from Start (S) to Goal (G).
Initialization:

- We begin by placing the start node S into our priority queue (Frontier) with a cumulative cost of 0.
- Frontier: [S(cost 0)]

- Explored Set: Empty.

Iteration 1:

- We pop the node with the lowest cost from the Frontier. This is node S.
- Goal Test: Is S the destination G? No.
- We expand node S by examining its connected neighbors. According to the graph data, S connects to A (cost 1) and G (cost 12).
- We calculate the cumulative costs and add these new paths to the Frontier, sorting them from lowest to highest cost.
- Frontier updates to: [A(cost 1, path S-A), G(cost 12, path S-G)]
- Explored Set: We add S.

Iteration 2:

- We pop the node with the lowest cost from the Frontier. This is node A (cost 1).
- Goal Test: Is A the destination G? No.
- We expand node A. Its neighbors are B (edge cost 3) and C (edge cost 1).
- We calculate cumulative costs: Path to B is (1 + 3) = 4. Path to C is (1 + 1) = 2. We insert these into the sorted Frontier.
- Frontier updates to: [C(cost 2, path S-A-C), B(cost 4, path S-A-B), G(cost 12, path S-G)]
- Explored Set: We add A. (Current explored: S, A).

Iteration 3:

- We pop the node with the lowest cost, which is currently C (cost 2).
- Goal Test: Is C the destination G? No.
- We expand node C. Its neighbors are D (edge cost 1) and G (edge cost 2).
- We calculate cumulative costs: Path to D is (cost to C + edge to D) = 2 + 1 = 3. Path to G is (cost to C + edge to G) = 2 + 2 = 4.
- We evaluate the Frontier. We have found a new path to the goal G with a cost of 4. There is already a path to G in the frontier with a cost of 12. Because UCS seeks the lowest cost, we completely discard the expensive path (cost 12) and replace it with our newly discovered optimal path (cost 4).
- Frontier updates to: [D(cost 3, path S-A-C-D), B(cost 4, path S-A-B), G(cost 4, path S-A-C-G)]
- Explored Set: We add C. (Current explored: S, A, C).

Iteration 4:

- We pop the lowest cost node, which is D (cost 3).
- Goal Test: Is D the destination G? No.
- We expand D. Its only forward neighbor is G (edge cost 3).
- We calculate cumulative cost: Path to G through D is (cost to D + edge to G) = 3 + 3 = 6.
- We evaluate the Frontier. We already possess a path to G with a cost of 4 (via S-A-C-G). Since this new path through D costs 6, which is higher, we discard the new path entirely.
- Frontier remains: [B(cost 4, path S-A-B), G(cost 4, path S-A-C-G)]
- Explored Set: We add D. (Current explored: S, A, C, D).

Iteration 5:

- We look at the Frontier. We have a tie between B (cost 4) and G (cost 4). The algorithm can pop either depending on tie-breaking rules, but if it pops B, B will expand to D (cost 4+3=7) which is discarded as D is explored. Let's assume it pops the goal node G.
- We pop G (cost 4).
- Goal Test: Is G the destination G? Yes!

- The search immediately terminates upon successfully testing a popped goal node.

Final Conclusion: By strictly adhering to the Uniform Cost Search protocol, the algorithm prioritized expanding cheaper local routes and successfully avoided the incredibly expensive direct route of S to G (cost 12). The final output is that the least-cost path from the starting warehouse S to the destination warehouse G is the sequence: S -> A -> C -> G. The total minimum cost required to transport packages along this network route is exactly 4 units (1 + 1 + 2).